ScienceDirect

Journal of Systems and Software

Volume 209, March 2024, 111919

Fine-grained smart contract vulnerability detection by heterogeneous code feature learning and automated dataset construction ☆

Jie Cai [a b], Bin Li [a], Tao Zhang [b], Jiale Zhang [a], Xiaobing Sun [a]

Show more

Share Cite

https://doi.org/10.1016/j.jss.2023.111919 ↗ Get rights and content ↗ View accessibility information

Highlights

- We propose a security best practice based dataset construction approach for vulnerability detection in Solidity smart contracts.
- We propose a code graph that merges AST subtrees into CFG nodes for heterogeneous code feature representation.
- We propose a deep learning model that combines GNN with the tree-L cell for heterogeneous code feature learning.
- The experimental results demonstrate the high effectiveness of our approach.

Abstract

Context:

Recently, several deep learning based smart contract vulnerability detection approaches have been proposed. However, challenges still exist in applying deep learning for fine-grained vulnerability detection in smart contracts, including the lack of the dataset with sufficient statement-level labeled smart contract samples and neglect of heterogeneity between syntax and semantic features during code feature learning.

Objective:

To utilize deep learning for fine-grained smart contract vulnerability detection, we propose a security best practices (SBP) based dataset construction approach to address the scarcity of datasets. Moreover, we propose a syntax-sensitive graph neural network to address the challenge of heterogeneous code feature learning.

Method:

The dataset construction approach is motivated by the insight that smart contract code fragments guarded by security best practices may contain vulnerabilities in their original unguarded code form. Thus, we locate and strip security best practices from the smart contract code to recover its original vulnerable code form and perform sample labeling. Meanwhile, as the heterogeneity between tree-structured syntax features embodied inside the abstract syntax tree (AST) and graph-structured semantic features reflected by relations between statements, we propose a code graph whose nodes are each statement's AST subtree with a syntax-sensitive graph neural network that enhances the graph neural network by a child-sum tree-LSTM cell to learn these heterogeneous features for fine-grained smart contract vulnerability detection.

Results:

We compare our approach with three state-of-the-art deep learning-based approaches that only support contract-level vulnerability detection and two popular static analysis-based approaches that support fine detection granularity. The experiment results show that our approach outperforms the baselines at both coarse and fine granularities.

Conclusion:

In this paper, we propose utilizing security best practices inside the smart contract code to construct the dataset with statement-level labels. To learn both tree-structured syntax and graph-structured semantic code features, we propose a syntax-sensitive graph neural network. The experimental results show that our approach outperforms the baselines.

Introduction

The smart contract is a program written by high-level programming languages (e.g., Solidity Dannen, 2017 or Vyper Buterin, 2018) and runs on the blockchain platform. As the smart contract has various properties, such as decentralized, unchangeable, and anonymous execution, it has been utilized in several areas (e.g., IoT, data management, or finance) (Zhou et al., 2022). However, as a program, the smart contract still suffers from vulnerabilities. Furthermore, as smart contracts often manage large amounts of cryptocurrency assets or sensitive information on the blockchain, vulnerabilities in these smart contracts can lead to serious consequences. For example, in the infamous theDAO incident (Falkon, 2017), hackers exploit a vulnerability in the smart contract code to steal approximately $70 million. Thus, the vulnerabilities become a severe threat to the security of the blockchain ecosystem. For the security of the blockchain, vulnerability detection in smart contracts has become an urgent and vital task and has received widespread attention from researchers.

As Solidity (Dannen, 2017) is the dominant programming language for smart contracts, most researchers select the smart contracts written by Solidity as the research object. To facilitate comparison and analysis, we also select the smart contracts written by Solidity as the research object to do the vulnerability detection for our approach. Specifically, several approaches (Luu et al., 2016, Tikhomirov et al., 2018, Brent et al., 2020, Schneidewind et al., 2020, Mossberg et al., 2019, Tsankov et al., 2018, Zhuang et al., 2020, Liu et al., 2021b, Liu et al., 2021a, Zhang and Liu, 2022, Nguyen et al., 2022, Wu et al., 2021, Zhang et al., 2022a) have been proposed to detect vulnerabilities for Solidity smart contract. These approaches can be divided into two categories: pattern-based and learning-based. The pattern-based static analysis approaches utilize some technology, such as symbolic execution (Luu et al., 2016) or taint analysis (Brent et al., 2020), to explore the potential execution space of a smart contract and check the existence of some predefined vulnerability patterns inside the code. However, some complex patterns and corner cases are non-trivial to be covered by the expert-defined patterns that make these approaches error-prone and high human effort cost. To Avoid relying on expert-defined patterns, researchers have proposed several learning-based approaches for Solidity smart contracts vulnerability detection. The general process of these approaches is converting the smart contract to a specific representation (e.g., sequence Wu et al., 2021 or graph Zhuang et al., 2020) and utilizing a deep-learning model to learn vulnerability-related code patterns from a well-labeled dataset. Compared with pattern-based approaches, learning-based ones can automatically extract vulnerability patterns

from prior vulnerable code instead of requiring expert involvement. However, existing learning-based approaches face several challenges when applied to fine-grained smart contract vulnerability detection. These challenges include:

Challenge 1: Lacking the Dataset with Sufficient Statement-Level Labels for Fine-Grained Smart Contract Vulnerability Detection. To achieve fine-grained smart contract vulnerability detection using learning-based approaches, a dataset with sufficient statement-level labeled smart contract samples is required to train the detecting model. However, the current dataset construction approaches, including manual-based labeling (Zhuang et al., 2020, Liu et al., 2021b, Liu et al., 2021a, Wu et al., 2021, Zhang et al., 2022a), existing detection tools-based labeling (Zhang et al., 2022b, Huang et al., 2022, Zhang et al., 2022a, Durieux et al., 2020), and vulnerability injection (Ghaleb and Pattabiraman, 2020, Feng, 2021), have limitations in generating such a dataset. For instance, manual-based labeling requires a high human effort with professional knowledge for manual inspection of many smart contracts. Furthermore, the detection tools-based labeling approach cannot generate statement-level labels due to the limitations of existing detection tools. The vulnerability injection-based approach only generates the samples by templates, which cannot fully cover the various forms of a given vulnerability type. Meanwhile, the injected vulnerable code is isolated from the original smart contract code context, which may not accurately reflect real-world smart contract scenarios. As a result, these three approaches cannot balance the effort required for labeling and the quality of statement-level labels.

Challenge 2: Neglecting the heterogeneity between code syntax and semantic features during code representation learning. Prior researchers (Yamaguchi et al., 2014, Zhou et al., 2019, Wang et al., 2020) have proposed using Code Property Graph (CPG) with Graph Neural Networks (GNN) to capture the syntax and semantic features inside the code. In CPG construction (Yamaguchi et al., 2014), it merges the Abstract Syntax Tree (AST) into the Program Dependence Graph (PDG) by converting each statement's AST subtree into a graph and attaching it under the corresponding node in the PDG. Then, these approaches utilize the GNN's message-passing paradigm to propagate information between nodes following edges in the CPG for feature learning. However, all these approaches neglect the heterogeneity between syntax and semantic features of the smart contract code. More specifically, syntax features are organized as a tree structure and embodied inside the AST, while semantic features are embodied inside the relations between statements and represented as a graph structure, such as the control flow graph (CFG). Furthermore, since the tree structure is hierarchical and organized from leaf to root, unifying these two heterogeneous features into a graph structure and learning features by aggregating information from each node's neighbors without following the original leaf-to-root order is inappropriate and breaks the original tree structure of the syntax features. Thus, the current common code feature learning approach cannot fully utilize the heterogeneous syntax and semantic features inside the smart contract code.

To solve the above challenges, we propose a novel approach utilizing the Security Best Practices (SBP) to construct the dataset with sufficient statement-level labels and a Syntax Sensitive Graph Neural Network (SS-GNN) model to address the heterogeneous syntax and semantic feature learning. **To address Challenge 1**, we propose to leverage the presence of security best practices in smart contract code for dataset construction. We have observed that the smart contract community has summarized several security best practices (SBP) for protecting smart contracts from vulnerabilities. These SBPs are widely accepted by the smart contract developer community and are commonly applied in current smart contract development. Our key insight is that the statements guarded by security best practices in a smart contract code reflect the knowledge of the developers about which statements may be vulnerable and require specific security measures to protect them. Based on this insight, we propose using the knowledge embodied in statements guarded by security best practices to construct the dataset. Specifically, this dataset construction approach involves locating code statements guarded by the security best practices in smart contract code and stripping the security best practices from them to restore their original, vulnerable code form and label them as vulnerable. In contrast, code fragments without any security best practices applied will be labeled as neutral. **To address Challenge 2**, we propose a novel approach that avoids simply merging the AST into the code graph during code representation construction. Instead, during code graph construction, we replace each node's content in the graph with each statement's corresponding AST subtree to maintain the original tree structure of the syntax feature. Then, during feature learning, we propose a syntax-sensitive GNN (SS-GNN) that incorporates an additional child-sum tree-LSTM cell (Tai et al., 2015) into the GNN model to enhance the model to capture syntax features from the tree structure before performing message passing between adjacent nodes to learn semantic features. Thus, compared with the

common GNN models that only capture the relations between nodes, our model can leverage the tree-structured syntax features inside each statement's AST subtree with the graph-structured semantic features in its context for smart contract vulnerability detection and named as Syntax Sensitive Graph Neural Network (SS-GNN). Moreover, we formulate the fine-grained smart contract vulnerability detection as the node classification task to train the detecting model.

We construct a dataset with 21,183 lines of vulnerable statements and 103,585 lines of neutral statements for statement-level smart contract vulnerability detection. We compare our approach against three state-of-the-art learning-based detection approaches (Zhuang et al., 2020, Wu et al., 2021, Zhang et al., 2022a) and two static analysis-based detection approaches (Tikhomirov et al., 2018, Feist et al., 2019). The result shows that our approach outperforms all baselines. We also compare our proposed SS-GNN with four widely used GNN models (Li et al., 2015, Kipf and Welling, 2016, Veličković et al., 2017, Brody et al., 2021), and the result shows that our SS-GNN outperforms these models in smart contract vulnerability detection tasks.

In summary, the main contributions of this article are as follows:

- We propose a dataset construction approach by utilizing the developers' knowledge embodied inside the security best practice to save human efforts.
- We formulate the fine-grained smart contract vulnerability detection task as a node classification task using a code graph, where each node represents a statement's AST subtree and propose a novel syntax-sensitive graph neutral network for heterogeneous code feature learning to identify suspicious vulnerable statements in smart contracts.
- We evaluate our approach, and the results show that our approach can effectively detect vulnerabilities over state-of-the-art vulnerability detection approaches at both statement and contract levels.

The rest of this article is organized as follows. In Section 2, we introduce the background of security best practices in smart contracts and explain the motivation of our approach. In Section 3, we explain the details of the dataset construction approach. Then, in Section 4, we explain our proposed vulnerability detection approach. In Section 5, we show the experiment setting and implementation of our approach. After that, we discuss the experimental results in Section 6. In Section 7, we discuss the performance and threats to the validity. Next, we discuss related studies in Section 8. Finally, Section 9 concludes this article.

Access through your organization

Check access to the full text by signing in through your organization.

Access through **your organization**

Section snippets

Background and motivation

In this section, we will provide some background knowledge about smart contract vulnerabilities and their corresponding security best practices. We will also discuss the motivation behind our proposed smart contract vulnerability detection framework. …

Security best practice based dataset construction

To overcome the limitations of existing dataset construction approaches, we propose utilizing security best practices within smart contract code to construct the dataset for smart contract vulnerability detection. Based on our observation, seasoned smart contract developers apply various security best practices to protect vulnerable code fragments. The presence of security best practices in a code fragment signifies the developer's knowledge of the potential vulnerability in the original, no …

Vulnerability detection approach

Fig. 11 shows the framework of our proposed vulnerability detection approach. Our approach takes the smart contract source code as input and proceeds to construct code graph representations for each function within the smart contract. The detection model conducts predictions for each node within the code graph, determining whether each node is vulnerable or not as the final output. More specifically, our approach consists of two phases: the training phase and the detection phase. The training …

Research questions

To evaluate our approach, we design experiments to answer the following research questions:

RQ1: How effective is our approach for detecting vulnerabilities in Solidity smart contracts compared with existing state-of-art smart contract vulnerability detection approaches?

The studies most relevant to our approach are deep learning-based approaches. However, most existing learning-based approaches only evaluate their performance in reentrancy vulnerability detection tasks for Solidity smart …

Performance comparison in coarse-grained vulnerability detection: Answer to RQ1

Table 4 shows the results of each vulnerability detection approach in terms of evaluating the metrics mentioned above for Reentrancy vulnerability detection at the contract-level. In general, our approach outperforms all other learning-based baseline approaches and pattern based baseline approaches. More specifically, our approach achieves a recall of 89.77%, improving the state-of-the-art by 23.80%, and a precision of 91.40%, improving the state-of-the-art by 31.34%. To the F1, ours achieves …

Performance analysis

In this paper, for deep learning-based fine-grained smart contract vulnerability detection, our approach can outperform baselines as ours can preserve the heterogeneity between tree-structured syntax features and graph-structured semantic features inside the code during feature learning, which is neglected by most related studies. For constructing a dataset with statement-level labeled smart contract samples, we propose utilizing the security best practices inside the smart contract code as the …

Related work

Smart contract vulnerability poses a serious threat to the security of the blockchain ecosystem. Existing Smart contract vulnerability detection approaches can be divided into the following categories: …

Conclusion

This paper proposes a security best practices based dataset construction approach with a code graph representation learning based approach for fine grained vulnerability detection in smart contracts. We propose to locate and strip the

code fragments guarded by security best practices as the vulnerable statement samples to construct our dataset. To learn the heterogeneous syntax and semantic features of the smart contract code, we have introduced a code graph where each node represents an AST …

CRediT authorship contribution statement

Jie Cai: Methodology, Software, Validation, Writing – original draft. **Bin Li:** Supervision. **Tao Zhang:** Writing – review & editing, Resources, Supervision. **Jiale Zhang:** Writing – review & editing, Conceptualization. **Xiaobing Sun:** Formal analysis, Validation, Project administration. …

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper. …

Acknowledgments

This work is supported by the National Natural Science Foundation of China (No. 62206238, No. 61972335), Natural Science Foundation of Jiangsu Higher Education Institutions of China (Grant No. 22KJB520010), Natural Science Foundation of Jiangsu Province (Grant No. BK20220562), the Six Talent Peaks Project in Jiangsu Province (No. RJFW-053), the Jiangsu "333" Project, the Yangzhou city-Yangzhou University Science and Technology Cooperation Fund Project (YZ202 1157, YZ2021158), the Natural …

Jie Cai received the Master's degree from Nanjing University of Post and Telecommunications, Nanjing, China, in 2016. He is working toward the Ph.D. in software engineering at Yangzhou University, Yangzhou, China. His research interests include blockchain security, software safety and security, etc. …

…
…

Recommended articles

References (42)

ZhangLejun *et al.*
Smart contract vulnerability detection combined with multi-objective detection
Comput. Netw. (2022)

Brent, Lexi, Grech, Neville, Lagouvardos, Sifis, Scholz, Bernhard, Smaragdakis, Yannis, 2020. Ethainter: a smart...

BrodyShaked *et al.*
How attentive are graph attention networks?
(2021)

BuiNghi D.Q. *et al.*
Infercode: Self-supervised learning of code representations by predicting subtrees

ButerinVitalik
Vyper documentation

(2018)

ConsenSys
Mythril
(2021)

ConsenSys
Solc-typed-ast
(2022)

DannenChris
Introducing Ethereum and Solidity, Vol. 1
(2017)

Durieux, Thomas, Ferreira, João F, Abreu, Rui, Cruz, Pedro, 2020. Empirical review of automated analysis tools on...

FalkonS.
The story of the DAO—Its history and consequences
(2017)

View more references

Cited by (29)

SolBERT: Advancing solidity smart contract similarity analysis via self-supervised pre-training and contrastive fine-tuning
2025, Information and Software Technology

Show abstract

Blockchain Smart Contract Security: Threats and Mitigation Strategies in a Lifecycle Perspective ↗
2025, ACM Computing Surveys

Securing Blockchain Systems: A Layer-Oriented Survey of Threats, Vulnerability Taxonomy, and Detection Methods ↗
2025, Future Internet

Enhancing Smart Contract Security with Explainable AI: A Framework for Re-entrancy Vulnerability Detection and Explanation ↗
2025, 2025 IEEE Systems and Information Engineering Design Symposium, SIEDS 2025

A Systematic Literature Review on Automated Software Vulnerability Detection Using Machine Learning ↗
2024, ACM Computing Surveys

Empirical Study of Move Smart Contract Security: Introducing MoveScan for Enhanced Analysis ↗
2024, ISSTA 2024 - Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis

View all citing articles on Scopus ↗

Jie Cai received the Master's degree from Nanjing University of Post and Telecommunications, Nanjing, China, in 2016. He is working toward the Ph.D. in software engineering at Yangzhou University, Yangzhou, China. His research interests include blockchain security, software safety and security, etc.

Bin Li is a professor in School of Information Engineering, Yangzhou University, China. His current research interests include software engineering, artificial intelligence.

Tao Zhang received the B.S. degree in automation, the M.Eng. degree in software engineering from Northeastern University, China, and the Ph.D. degree in computer science from the University of Seoul, South Korea. After that, he spent one year with the Hong Kong Polytechnic University as a postdoctoral research fellow. Currently, he is an associate professor with the School of Computer Science and Engineering, Macau University of Science and Technology (MUST). Before joining MUST, he was the faculty member of Harbin Engineering University and Nanjing University of Posts and Telecommunications, China. He is a senior member of IEEE and ACM.

Jiale Zhang received the Ph.D. degree in computer science and technology the College of Computer Science and Technology, Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2021. He is currently the Associate Professor with the School of Information Engineering, Yangzhou University, Yangzhou, China. His research interests are mainly federated learning, blockchain security, and privacy-preserving.

Xiaobing Sun is currently a professor in School of Information Engineering, Yangzhou University, China. He received the Ph.D. degree in School of computer science and engineering, Southeast University in 2012. His research interests include intelligent software engineering and software data analytics.

☆ Editor: Yan Cai.

View full text

© 2023 Elsevier Inc. All rights reserved.

ELSEVIER

All content on this site: Copyright © 2026 Elsevier B.V., its licensors, and contributors. All rights are reserved, including those for text and data mining, AI training, and similar technologies. For all open access content, the relevant licensing terms apply.

RELX™